

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA3711

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Code of Conduct:

I S.Kamalesh (192521013) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.KAMALESH

(192521013)

Question 1: A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

Answer:

Introduction: Testing is not something that can be done casually after coding is over. It is an engineering activity, which means it has to be planned, designed, executed, measured and improved just like any other engineering work. In the banking application the failures are frequent mainly because the team tests whatever comes to mind, without any plan, without written test cases and without any measure of how much has been covered. So the first step is to replace this unstructured way with a disciplined test engineering process.

Systematic test design: I would first study the requirements (for example fund transfer, balance enquiry, bill payment and login) and prepare a test plan that states the scope, the risks, the entry and exit criteria and the schedule. After that, test cases are designed from each requirement using proper techniques and every test case carries an id, the input, the expected result and the requirement it traces back to. A traceability matrix is kept so that no requirement is left without a test.

Tools and standards: Tools bring discipline and speed. A test management tool such as TestLink or Jira with Zephyr can store test cases and results, Selenium can automate the repeated regression checks of the web banking screens, JMeter can be used for load checks and Postman for the service level checks. On the standards side the team can follow IEEE 829 for the test documents and IEEE 1044 for the defect classification, so that everyone writes and reads the same format.

Techniques to replace ad-hoc testing: Equivalence class partitioning and boundary value analysis for amount fields, decision tables for the transfer rules (daily limit, minimum balance, account status), state transition testing for the session and login lock, and use case based testing for the complete customer flow. Along with these, error guessing is still used but only as a supplement and not as the main method.

Effect on defect detection and quality: Since every requirement is mapped to test cases, the coverage becomes measurable and the missed areas are visible early. Defects are found in the early phases, they are logged with clear details and the same tests can be rerun after every fix. Because of this the number of defects that escape to production reduces and the quality of the product becomes predictable.

Practical example: Take the fund transfer module where the daily limit is Rs. 50,000. Under ad-hoc testing a tester may try only 1,000 or 10,000 and declare it passed. With boundary value analysis the test inputs would be 0, 1, 49,999, 50,000 and 50,001, and a decision table checks the combinations of low balance and blocked account. The case of 50,001 being accepted wrongly, which random testing would have missed, is caught immediately. This justifies the engineering approach.

Question 2: An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.

Answer:

Introduction: An ATM works correctly only when every stage of its development is verified in a defined way. The inconsistent results here come from the absence of a process, so I would build the testing process in parallel with the SDLC phases, following the idea of the V-model where each development phase has a matching test activity.

Stages and their relevance: Requirement phase: testers review the requirements for clarity and testability and prepare the acceptance test plan. This avoids ambiguous requirements which are the origin of many defects. Design phase: the system and integration test plans are prepared and the design is reviewed, for example the interaction between the card reader, the PIN module and the bank server. Coding phase: unit tests are written and code reviews are done by the developers. Integration phase: the modules are combined and interface defects such as a wrong message format between ATM and host are checked. System testing phase:

the complete ATM is tested against the requirements including performance and security. Acceptance phase: the bank verifies the system with real-like scenarios before release. Maintenance phase: regression tests are run whenever a change is made.

Application to ATM withdrawal: For the withdrawal function the test cases would cover valid card and correct PIN, wrong PIN three times (card blocked), amount more than balance, amount not in multiples of 100, amount above the daily limit, cash dispenser having insufficient notes, network failure after the amount is deducted but before cash is given, and the receipt printing. Each case has a defined expected result such as the correct message and the correct updated balance.

How consistency reduces defects: When the same process is followed every time, the same checks are made for every release, nothing depends on the memory of an individual tester, and results are repeatable. The defects are found in the phase in which they were introduced where the cost of fixing is very small, instead of being found after deployment.

Expected outcomes: The outcome is a reduction in inconsistent behaviour, fewer production failures, complete traceability from requirement to test, measurable coverage, and higher confidence of the bank and the customers in the ATM system.

Question 3: A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.

Answer:

Definitions: An error (mistake) is a human action that produces an incorrect result, for example a programmer using the wrong condition. A defect (fault or bug) is the flaw present in the code or document because of that mistake. A failure is the visible incorrect behaviour of the software when the defective code is executed. Test cases, the expected and actual output and the test oracle are what let us notice the failure.

Application to the web application: In the login case, suppose the developer writes the check as username = password instead of comparing with the stored password. This wrong thinking is the error, the wrong statement in the code is the defect, and when a user enters a wrong password and still gets logged in, that is the failure. In the transaction case, if the developer forgets to subtract the service charge, the missing statement is the defect and the mismatch between the expected balance and the displayed balance is the failure.

Relationship: The relation is a chain: error leads to a defect in the work product, and a defect leads to a failure only when the defective part is executed under suitable conditions. So a defect may remain hidden and never cause a failure, but every failure is traced to at least one defect.

Effect of improper understanding: If the team uses these words loosely, the defect reports become confusing. A tester may report only the symptom and the developer may fix the symptom and not the real cause, so the defect comes back. Time is wasted in arguments, the metrics become wrong and the root cause analysis is not possible.

Improving defect reporting clarity: Every report should have a unique id, a short title, the module, the steps to reproduce, the input data, the expected result, the actual result, severity and priority, the environment and a screenshot or log. The report should describe the failure clearly and, when known, point towards the defect location. For example: "Login accepts invalid password 'abc123' for user ravi01; expected message: Invalid credentials." This is precise and the developer can reproduce it at once.

Question 4: A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.

Answer:

Introduction: The observation that most defects are in a few modules is a known pattern and it is exactly what the testing principle of defect clustering says. The remaining principles also help the team to plan its effort better.

Principles that guide test planning: Defect clustering: roughly 80 percent of the defects are found in 20 percent of the modules, so these modules get more test effort. Testing shows the presence of defects, not their absence, so we cannot claim the software is defect free. Exhaustive testing is impossible, so testing is based on risk and priority. Early testing saves time and cost because the defects found in requirements and design are cheap to fix. The pesticide paradox says that repeating the same tests will stop finding new defects, so the test cases must be reviewed and updated regularly. Testing is context dependent, and absence of errors is a fallacy, meaning that a defect free product is of no use if it does not meet the user needs.

Prioritizing test cases: The test cases are ranked using risk (probability of failure multiplied by impact), the defect history of the module, the complexity of the code, the frequency of use by customers and the recent changes. Test cases for high-risk and defect-prone modules are executed first and in more depth, then the medium ones, and the low risk ones last. A small smoke suite is run first after every build.

Improved efficiency: The team spends its limited time where the defects actually are, the critical failures are found earlier, the schedule delay reduces, and the cost of testing comes down without lowering the quality.

Real-time example: In an online shopping project, the defect log showed that the payment and the discount modules had 65 of the 100 defects. The tester added more boundary, negative and integration tests to these

two modules, ran them first in each cycle and refreshed the test data in each release. The result was that the serious defects were found in the first week of testing instead of the last, and the release was not delayed.

Question 5: A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

Answer:

Introduction: A defect repository is a database in which all the defects are stored along with their details. It is of value only when it is analysed. Here the company stores the data but does not use it, so the same defects appear again and again in every release.

How defects originate: Defects originate in every phase of the life cycle. In the requirement phase they come from missing or ambiguous requirements, in the design phase from wrong logic or interface design, in the coding phase from syntax, logic and data handling mistakes, and in the testing phase from wrong test cases or wrong test environment. Poor communication, lack of knowledge and time pressure are the human reasons behind them.

Categorization: As per the defect classification, defects are grouped into requirement and specification defects, design defects (algorithmic, control and logic, data, module interface and external interface defects), coding defects (control, logic and sequence, data, initialization, interface and documentation defects) and testing defects. In the repository each defect is also tagged with its severity, priority, module, phase of origin, phase of detection, root cause and status.

Using repository data: The stored data can be analysed to find which defect classes are most frequent, which modules are weak and in which phase the defects are being introduced. If many defects are of the data initialization type, the test design adds more cases on initial values. Test cases can also be created directly from old defects to form a regression suite, so that a corrected defect is never reintroduced. The same data supports prevention, for example through checklists for code reviews and improved coding guidelines.

Improvements to avoid recurrence: Conduct root cause analysis for every major defect, review the repository at the end of each release, keep a lessons-learned list, train the team on the top defect classes, update checklists, and link every fixed defect with a regression test.

Expected improvements: Fewer repeated defects, better test coverage in the weak areas, shorter fix time, better predictability of quality, and a gradual shift from defect detection to defect prevention.

Question 6: A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.

Answer:

Finding the root cause: The crash during the fund transfer should be handled as a failure to be traced back to its defect. I would first reproduce it with the exact steps, device, OS version, network type and data, and check the application logs and crash reports. Then the possible causes are narrowed down: unhandled exceptions when the network drops in the middle of a transaction, memory leaks, null values in the beneficiary data, wrong handling of large amounts or decimal values, timeouts from the server, and double tapping on the transfer button. Debugging with the log traces then locates the faulty code.

How systematic testing helps: When the testing is planned, the transfer function is covered by test cases from different views: functional (valid and invalid account, amount and OTP), boundary (minimum and maximum limit), negative (empty fields, special characters), interruption (call or message during transfer, loss of network), compatibility (different devices and OS) and performance and stress tests. Failures of this kind are therefore found in the test lab and not by customers.

Example: Suppose the application crashes when the beneficiary name contains an apostrophe such as O'Brien. A negative test with special characters would reveal it. Likewise a test where the internet is switched off after pressing the Pay button would show if the app fails to handle the timeout and whether the amount is debited twice.

Improvements in testing practices: Add automated regression tests for the transfer flow, use a device farm to test on many real devices, include non-functional testing, introduce stress and endurance tests, do code reviews of exception handling, monitor crashes after release with tools such as Firebase Crashlytics, and keep a defect repository so that the crash patterns are learned.

Question 7: A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

Answer:

Testing: Testing is the process of executing the software with chosen inputs to find out whether it behaves as expected. It shows the presence of failures and is normally done by the testers, who need not know the internal code. Its goal is to detect and report defects and to give information about quality.

Debugging: Debugging begins after a failure has been found. It is the process of locating the exact defect in the code, finding its cause and correcting it. It is done by the developers, it needs knowledge of the code, and its goal is to remove the defect. After the correction, the software goes back for retesting.

Roles in software quality: Testing measures quality and gives evidence about the problems, while debugging improves quality by removing the problems. Both are needed, and one cannot replace the other. Testing is planned and can be scheduled, whereas debugging is a reaction to a reported failure and its time cannot be predicted exactly.

Effect of improper usage: If developers think that their own debugging is testing, then only the visible and expected paths are tried and most defects are not detected. If the testers try to debug, their time is lost in code analysis and the independent view is lost. The defects get fixed without proper reports, retesting is skipped and regression defects appear.

Structured integration: A good approach is this: the tester executes the planned test cases, logs the failure with the steps and data, the developer reproduces it, debugs and fixes it, the tester retests the fix and runs regression tests, and then the defect is closed in the repository. Responsibilities are separated clearly and information flows through the defect tracking tool, so both processes support each other.

Question 8: An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

Answer:

Strategy for cross-browser testing: First the target browsers and versions are decided from the analytics of the e-commerce site, for example Chrome, Edge, Firefox and Safari on desktop and Chrome and Safari on mobile, together with the operating systems and screen sizes. A compatibility matrix is prepared. Then the critical user journeys (search, product page, cart, checkout and payment) are listed and test cases are written for them. Functional behaviour, layout and appearance, JavaScript behaviour, form validation, cookies and session handling, and performance are checked on every browser in the matrix.

How such defects originate: The browsers use different rendering engines and each interprets HTML, CSS and JavaScript slightly differently. Defects come from browser specific or non-standard code, unsupported CSS properties or APIs, differences in default styles, fonts, and cookie or storage policies, outdated browser versions, and responsive design issues. If the developers test only on one browser, the other browser problems are not discovered.

Systematic testing and reliability: With a defined matrix and fixed test cases, every release is checked in the same manner, so the inconsistent behaviour is found before deployment. Reliability grows because the failing combinations are recorded and re-tested for each release.

Tools and techniques: Selenium WebDriver and Selenium Grid for automating the same test across browsers, BrowserStack or LambdaTest for real devices and browsers in the cloud, Cypress or Playwright for modern web testing, and Chrome DevTools for debugging layout. Techniques such as equivalence partitioning to reduce the browser combinations, pairwise testing for the combinations of browser, OS and device, visual comparison of screenshots and a regression suite for each release give better coverage.

Question 9: A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

Answer:

Classification of the defects: Incorrect salary calculation caused by logical errors falls under coding defects, in the sub-class of control, logic and sequence defects. Examples in the payroll system are: using the wrong operator (basic + HRA - PF written as basic + HRA + PF), wrong order of calculation so that tax is applied before the deduction, wrong loop condition so that the last employee is skipped, and a wrong condition in the overtime rule ($>$ instead of $\geq$). If the formula itself was specified wrongly, the defect is classified as a requirement or design defect, and if numbers are not initialised or are rounded wrongly it is a data defect.

How understanding defect classes improves testing: When the class of defect is known the tester can choose a suitable technique. For logic defects, decision tables, boundary value analysis and path testing are best. For data defects, tests of initial values, data types and rounding are added. The classes also show where the defects are concentrated, so the test effort can be focused.

Prevention in future releases: Use a clear written specification of every salary rule, conduct formula and code reviews and inspections with a checklist for logic errors, write unit tests for each calculation, keep a regression suite containing the earlier salary defects, and compare the system results with manually calculated sample salaries (oracle). Static analysis tools can also be used to find suspicious conditions.

Improvements in testing strategy: Add boundary tests for the salary slabs, test combinations of allowances and deductions using decision tables, run parallel testing with the old payroll results, automate the monthly payroll regression, and review defect data after each release so that the strategy is refined.

Question 10: A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Answer:

Why random testing is not enough: Executing random tests means nothing is planned: some functions are tested many times while others are never touched, the tests cannot be repeated or reviewed, and nobody knows when to stop. Applying testing as an engineering activity means that test cases are designed before execution in a planned and documented manner.

Importance of structured test case design: A designed test case has an id, objective, preconditions, input, expected output, and the postconditions. Its importance is that it gives coverage of all requirements, repeatability for regression, easy review by others, traceability to the requirement, measurable progress and an objective result. It also makes it possible to train new testers and to automate.

How it improves defect detection: Test design techniques are built to produce inputs which have a high probability of revealing defects. For instance, in an age field accepting 18 to 60, random testing may use 25 and 30, which pass. Boundary value analysis uses 17, 18, 19, 59, 60 and 61 and quickly shows an off-by-one defect. Therefore the number of defects found per test case is larger and the important defects are found sooner.

Techniques to enhance effectiveness: Black box techniques such as equivalence class partitioning, boundary value analysis, decision tables, state transition testing and cause-effect graphing; white box techniques such as statement, branch and path coverage; experience based techniques such as error guessing and exploratory testing in a time-boxed form; and peer reviews of the test cases. A requirement traceability matrix and coverage measurement complete the approach.

Question 11: A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

Answer:

Analysis of the problem: In the healthcare application the same patient data is showing differently in different modules, such as registration, billing, pharmacy and laboratory. This points to defects in the integration and data handling, so I would trace one patient record across the modules and compare the values, formats and timing of updates to locate where the data diverges.

How defects originate: They may originate from unclear requirements about which module owns the data, a design with duplicate data stores, different data types or formats in different modules (for example date format or units), missing validation, wrong update logic where a change is not propagated to the other modules, and

concurrency problems when two modules write the same record. Lack of communication between the teams building the modules is the usual human cause.

Systematic testing for data integrity: Integration testing checks the interface and data flow between the modules. Data validation tests check the formats, ranges and mandatory fields. CRUD tests check that create, update and delete in one module are reflected in the others. Database consistency checks compare the records using queries, and end-to-end scenarios (admit patient, prescribe, bill, discharge) follow the data through the entire flow. Negative tests and concurrency tests complete this. In healthcare, where wrong data can affect treatment, the traceability and the security of the data are also tested.

Improvements in tracking and reporting: Use a defect tracking tool, record the module, the data involved and the steps with screenshots, classify severity high for patient safety issues, link every defect to the requirement and the test case, review the open defects daily, and analyse the repository to find the repeating data inconsistencies.

Question 12: A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

Answer:

Analysis using the defect repository: Defects repeat in spite of testing, which suggests that the lessons from the earlier defects are not being used. The repository holds the history of every defect and so it is the best source for the analysis. I would extract the data of the last few releases and check which defects reappear, in which modules, of what class and in which phase they were introduced.

How repository data supports test design: The recorded defects show the weak areas and the types of inputs that caused failures, so new test cases can be designed for them. Every closed defect is converted into a regression test. The high-risk modules identified from the data are given more tests, and the test priority is decided by the history. The data also shows if the test cases were missing or were wrong, which is the basis for improving the test design.

Identifying defect patterns: Defects are grouped by module, class, severity, origin and root cause, and then Pareto charts and trend charts are prepared. For example, the chart may show that 70 percent of the defects are in the report module and that most are of the boundary type. Repeated defect descriptions after a fix, or defects reopened many times, indicate an incomplete fix or missing regression tests.

Suggestions to prevent recurrence: Perform root cause analysis for repeated defects, add the regression test for each fixed defect, update the review checklists and coding standards, give developers feedback on the common defect types, set up a regular defect review meeting before each release, and track the defect trend as a measure of improvement.

Question 13: A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

Answer:

Gaps in testing: The learning system fails when many students log in at the same time, which means the performance, load and stress scenarios were never tested. The testing was probably limited to functional checks with a single user. Other gaps are no realistic data volume, no test of the concurrent sessions and quiz submissions, no check of the server and database limits, and no monitoring.

Effect of test planning and coverage: A test plan should cover non-functional requirements like response time, throughput and the number of users supported, not only features. If the plan does not include them, the coverage on that side is zero and the weakness remains hidden until real usage. Good coverage of workload scenarios such as login rush, video streaming, and exam submission time gives confidence about the performance.

Better strategy and reliability: Define performance goals (for example 5,000 users with response under 3 seconds), prepare workload models from the actual usage, and execute load testing, stress testing to find the breaking point, spike testing for sudden increase, and endurance testing for long durations. The results are analysed to find bottlenecks such as slow queries or insufficient memory and the fixes are re-tested. This makes the system stable and predictable at the peak.

Enhancements: Use tools like JMeter, LoadRunner or Gatling, test in an environment similar to production, introduce auto scaling and caching, use monitoring tools to watch CPU, memory and response time, run performance tests in every release, and keep a capacity plan before the exam seasons.

Question 14: A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

Answer:

Role of the tester in the SDLC: The tester is not only the person who executes tests at the end. In each phase there is a role: reviewing the requirements for testability and completeness, preparing the test plan and design along with the design phase, reviewing the code or checking unit tests, executing the integration and system tests, and supporting acceptance and maintenance testing. When the testers are brought in only after development, this role is lost.

Impact of early involvement on quality: Defects introduced in the requirement or design phase are much cheaper to fix when found early, and the cost of a defect rises sharply in the later phases. Early involvement

also gives the testers a good understanding of the product, ambiguity in requirements is cleared at once, the test cases are ready by the time the build arrives, and the total schedule becomes shorter.

Practical example: In an online ticket booking project the tester, while reviewing the requirement, noticed that the maximum number of seats per booking and the behaviour when the payment times out were not defined. The question was raised in the review itself and the requirement was corrected. Had it been found after coding, the booking and payment modules would have needed rework.

Changes in team collaboration: Include testers in requirement and design reviews, adopt agile or shift-left practices where the developers and the testers work together in the same sprint, share the test cases with developers, hold daily meetings, use a common defect tracking tool, make developers and testers jointly responsible for quality, and encourage pair work for reviews and unit tests.

Question 15: A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

Answer:

Identifying the gaps: The system gives correct results for some inputs and fails for others, which shows that the tested inputs were only a small part of the input domain. To find the gaps I would map the requirements to the test cases using a traceability matrix and measure the code coverage (statement, branch and condition) with a coverage tool. Requirements without tests and code branches that were never executed are the gaps.

How coverage ensures reliability: Coverage tells us how much of the requirement, the code and the input domain has been exercised. High coverage means fewer untested paths where hidden defects can remain, so the reliability of the product increases. It should be remembered that 100 percent coverage does not prove absence of defects but a low coverage always shows risk.

Improving the test case design: Partition the inputs into equivalence classes and add at least one test for each valid and invalid class, add boundary tests, use a decision table for combinations of conditions, add state transition tests where behaviour depends on history, and for the white box side add tests until all branches and the important conditions are covered. For example, for a discount rule with membership and amount conditions, a decision table gives all the combinations which random tests would miss.

Techniques for better validation: Black box techniques (equivalence classes, boundary values, decision tables), white box techniques (statement, branch, path coverage), error guessing, reviews and inspections of the test cases, mutation testing for the quality of the tests, and regression testing after each change.

Question 16: A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

Answer:

Improving defect reporting: When reporting is poor, the developer cannot reproduce the problem, so many cycles are lost asking for details. Applying testing concepts, the failure (what was observed) is recorded exactly against the expected result, and a standard defect report format is used for all the defects.

Clear definitions and tracking: If the terms error, defect and failure and the terms severity (impact on the system) and priority (urgency of the fix) are defined and used in the same way by all, then defects can be correctly classified and routed. A defect gets a lifecycle status such as New, Assigned, Open, Fixed, Retest, Verified, Closed and Reopened, and the tracking shows exactly at which stage each defect is and who is responsible.

Structured reports and communication: A structured report has the defect id, title, module, version, environment, steps to reproduce, expected and actual results, severity, priority, attachments such as screenshots and logs, the reporter and the date. For a logistics application an example is: "Shipment status not updated to Delivered after proof of delivery upload; expected status Delivered; actual status In Transit; Build 2.3, Chrome." This gives developers, testers and the managers the same understanding and removes back and forth.

Improvements in reporting standards: Adopt an IEEE 829 or IEEE 1044 based template, use a tool such as Jira or Bugzilla, make the main fields mandatory, give guidelines and training on writing a good report, report defects immediately after finding them, review the reports for quality, and publish regular defect summary reports and metrics to the management.

Question 17: A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

Answer:

Process-based approach: Inconsistent testing across the modules means that each team is following its own way. The remedy is to define one standard test process for the whole project and to make everyone follow it. A model such as the Test Maturity Model (TMM) or TMMi can be used to assess and improve the maturity of the process step by step.

How standardized processes improve quality: A standard process gives the same stages, templates, entry and exit criteria and defect reporting format for all the modules. The results become comparable, the effort

can be estimated and the weaknesses can be found. The dependence on individuals reduces and new team members learn faster, so the overall quality becomes uniform.

Example: In a college management system the admission, examination and fee modules were tested by three teams in different ways. The project introduced a common process: test planning with a standard template, test design using the same techniques, a common test case format, uniform entry and exit criteria, and one defect tracker. After this, the coverage and the defect data of the three modules could be compared, and the examination module, which had a low coverage, was easily identified and strengthened.

Methods to ensure consistency: Prepare a test policy and a test process document, supply templates and checklists, do peer reviews and audits of test artefacts, appoint a test lead or test process group to monitor compliance, hold regular training, use common tools and a single repository, and measure through metrics such as coverage and defect density so that the process is continuously improved.

Question 18: A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

Answer:

Why the edge cases were missed: Edge cases are the inputs at the limits of the allowed range or the unusual conditions, such as the minimum and maximum values, empty input, very long text, zero, negative numbers or the last day of the month. They are missed when the testers use only typical values, the requirements do not state the limits, the time is short, and the test design techniques are not applied. Testing then confirms only the normal behaviour.

How boundary testing helps: Defects are very common at the boundaries because of mistakes such as using $<$ in place of $<=$. Boundary value analysis chooses the test inputs at and around the limits: the minimum, just above the minimum, the nominal value, just below the maximum and the maximum, and also just outside the valid range. A small number of tests therefore catch many defects.

Including edge cases in testing: Identify every input field and its limits from the requirements, list the boundaries, and prepare tests for them. Example: for a password length of 8 to 12 characters the tests use 7, 8, 9, 11, 12 and 13 characters. Also test null or blank input, special characters, maximum file sizes, leap year dates, zero quantity and large numbers, and add them to the regression suite.

Improvements in test design: Use boundary value analysis along with equivalence partitioning, ask for clearly stated limits in the requirements, hold review meetings to spot missing cases, use error guessing and exploratory testing based on past defects, check the coverage of the limits through a checklist and keep a record of the edge case defects in the repository for future reference.

Question 19: A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

Answer:

Addressing the problem: When defects are not documented, nobody knows what has been found, who is fixing it, or whether the same defect was reported before. To address this, a defect repository, which is a central database of all the defects, should be established and made compulsory for all the team members.

How documentation improves defect management: Documentation gives a single reliable record of every defect with its history. The status and ownership are visible, the duplicates are avoided, the developers get the information necessary to reproduce the problem, the managers can track the progress and decide on release, and the information is not lost when people leave the project.

How structured repositories support analysis: Because each defect is stored in a fixed format with fields such as module, class, severity, priority, origin phase, detection phase, root cause and status, the data can be sorted and charted. We can find the defect density of each module, the trend over the releases, the most common classes and the average fixing time. This analysis then supports test planning, process improvement and prevention of defects.

Improvements in documentation practices: Adopt a standard template (based on IEEE 1044), use a tracking tool like Jira or Bugzilla, make the important fields mandatory, define the defect life cycle and status rules, train the testers to write precise descriptions with steps and evidence, check the quality of the reports regularly, and review the repository in periodic meetings.

Question 20: A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.

Answer:

Redesigning the test approach: The repeated failures in transaction processing show that testing was improper and incomplete. I would redesign the approach as a structured engineering process: analyse the requirements and risks, prepare a master test plan, design test cases with proper techniques, set up a test environment similar to production, execute and log results, and report and track defects through to closure with regression testing.

How structured testing improves quality: Every requirement of the transaction processing is covered by test cases, the testing is repeatable, the defects are found early and the status is visible through metrics. The risk of failures in production comes down and the confidence grows with each release.

Examples: Functional tests: transfer between accounts with valid and invalid data; the debited and credited amounts must match. Boundary tests: amounts such as 0, 1, the limit and limit plus one. Negative tests: insufficient balance, blocked account, wrong OTP. Transaction integrity tests: if the system fails after debit but before credit, the transaction must be rolled back (the ACID properties). Concurrency tests: two simultaneous withdrawals from the same account must not make the balance negative. Security and performance tests: protection against unauthorised access and behaviour with a large number of transactions.

Techniques to enhance defect detection: Equivalence partitioning, boundary value analysis, decision tables and state transition testing for the design; risk based testing to give priority to the critical flows; automation of regression with tools like Selenium; reviews and inspections of the requirements and the code; use of the defect repository to learn from past failures; and root cause analysis with preventive actions.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand